Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

КТ № 4.

Согласно ТЗ, для тестирования был взят публичный API «Petstore» доступный по адресу https://petstore.swagger.io/.

В рамках выполнения данной КТ был установлен пакет requests через pip install.

```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> pip install requests
```

На основе примера из ТЗ был создан файл base_request.py, в котором был реализован класс BaseRequest, содержащий в себе функции для отправки и дальнейшей обработки GET/POST/PUT/DELETE запросов.

```python
def get(self, endpoint, endpoint_id, expected_error=False):
    url = f'{self.base_url}/{endpoint}/{endpoint_id}'
    response = self._request(url, 'GET', expected_error=expected_error)
    return response.json()

def post(self, endpoint, endpoint_id, body):
    url = f'{self.base_url}/{endpoint}/{endpoint_id}'
    response = self._request(url, 'POST', data=body)
    return response.json()

def put(self, endpoint, endpoint_id, body):
    url = f'{self.base_url}/{endpoint}/{endpoint_id}'
    response = self._request(url, 'PUT', data=body)
    return response.json()

def delete(self, endpoint, endpoint_id):
    url = f'{self.base_url}/{endpoint}/{endpoint_id}'
    response = self._request(url, 'DELETE')
    return response.json()
```

На основе данного класса было реализовано тестирование сущностей User и Store, к каждой из них было создано не менее, чем по 4 различных запросов.

Сущность User	Сущность Store
1. Запрос для создания новой записи (методом POST); 2. Запрос для получения ранее созданной записи (методом GET); 3. Запрос для изменения ранее созданной записи (методом PUT); 4. Запрос для удаления ранее созданной записи (методом DELETE); 5. Запрос для получения ранее удаленной записи, с целью	1. Запрос для создания новой записи (методом POST); 2. Запрос для получения ранее созданной записи (методом GET); 3. Запрос на удаление ранее созданной записи (методом DELETE); 4. Запрос на получение всех существующих записей в

Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

убедиться в удалении (методом GET).	подмаршруте эндпоиндта store/inventory (методом GET);

На данном этапе запуск запросов осуществлялся напрямую через Python без использования фреймворка для автоматизированного тестирования.

```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> python base request.py
```

Результаты выполнения запросов отображались в консоли, что позволило проверить корректность работы методов API.

```
'POST example'
'https://petstore.swagger.io/v2/user/'
200
'OK'
'{"code":200,"type":"unknown","message":"101"}'
{'code': 200, 'message': '101', 'type': 'unknown'}
'***********'
'GET example'
'https://petstore.swagger.io/v2/user/esket1'
200
'OK'
'{"id":101,"username":"esket1","firstName":"Dan","lastName":"Baykov","email":"baykovdr23@st.ithub.ru","p
5","phone":"1234567890","userStatus":1}'
{'email': 'baykovdr23@st.ithub.ru',
 'firstName': 'Dan',
 'id': 101,
 'lastName': 'Baykov',
 'password': '12345',
 'phone': '1234567890',
 'userStatus': 1,
 'username': 'esket1'}
'***********'
'PUT example'
'https://petstore.swagger.io/v2/user/esket1'
200
'OK'
'{"code":200,"type":"unknown","message":"101"}'
{'code': 200, 'message': '101', 'type': 'unknown'}
'***********'
'DELETE example'
'https://petstore.swagger.io/v2/user/esket1'
200
'OK'
'{"code":200,"type":"unknown","message":"esket1"}'
```

Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

```
'GET example'
'https://petstore.swagger.io/v2/user/esket1'
'404
'Not Found'
'{"code":1,"type":"error","message":"User not found"}'
{'code': 1, 'message': 'User not found', 'type': 'error'}
'**********'
'POST example'
'https://petstore.swagger.io/v2/store/order/'
'200
'OK'
'{"id":9999,"petId":1,"quantity":1,"shipDate":"2026-02-25T10:00:00.000+0000","status":"placed","complete":true}'
{'complete': True,
 'id': 9999,
 'petId': 1,
 'quantity': 1,
 'shipDate': '2026-02-25T10:00:00.000+0000',
 'status': 'placed'}
'**********'
'GET example'
'https://petstore.swagger.io/v2/store/order/9999'
'200
'OK'
'{"id":9999,"petId":1,"quantity":1,"shipDate":"2026-02-25T10:00:00.000+0000","status":"placed","complete":true}'
{'complete': True,
 'id': 9999,
 'petId': 1,
 'quantity': 1,
 'shipDate': '2026-02-25T10:00:00.000+0000',
 'status': 'placed'}
'**********'
'DELETE example'
'https://petstore.swagger.io/v2/store/order/9999'
'200
'OK'
'{"code":200,"type":"unknown","message":"9999"}'
{'code': 200, 'message': '9999', 'type': 'unknown'}
'**********'
```

Примечание: в отчете по GET запросу, отмеченным красным цветом, была выдана ошибка 404 (Not Found) в силу того, что этот запрос нацелен на проверку успешности выполнения предыдущего запроса, а именно запроса методом DELETE для удаления записи. Следовательно, это и есть ожидаемая реакция и фактической ошибки нет.

```
'GET example'
'https://petstore.swagger.io/v2/store/inventory'
'200
'OK'
('{"placed":2,"string":551,"sold '
 '":1,"desconocido":1,"pending":50,"available":263,"weisskeiner1":1,"soldout":1,"notavailable":1,
d":1,"available, '
 'sold":1,"mjnvxupkf@gmail.com":1,"invalid-status":1,"sold":30,"availableee":2,"NOT '
 'available":1,"d":1,"awaiable":1,"availabl":1,"peric":14,"Ö\\u0007D":1,"Active":1,"8":1,"availbl
{'8': 1,
 'Active': 1,
 'NOT available': 1,
 'Sold': 1,
 'availabl': 1,
 'available': 263,
 'available, sold': 1,
 'availableee': 2,
 'availble': 1,
 'awaiable': 1,
 'd': 1,
 'desconocido': 1,
 'invalid-status': 1,
 'mjnvxupkf@gmail.com': 1,
 'nonavailable': 2,
 'notavailable': 1,
 'pending': 50,
 'peric': 14,
 'placed': 2,
 'sold': 30,
 'sold ': 1,
 'soldout': 1,
 'string': 551,
 'weisskeiner1': 1,
 'Ö\x07D': 1}
'**********'
{'8': 1,
 'Active': 1,
 'NOT available': 1,
```

Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

КТ № 5.

Для выполнения данной КТ был взят файл base_request.py, созданный в ходе выполнения КТ № 4.

base_request.py

В рамках выполнения КТ5 был установлен фреймворк pytest для автоматизации тестирования, а также библиотеки Pydantic и Allure-Pytest через pip install. Для генерации HTML-отчета был установлен Allure Commandline через npm.

```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> npm install -g allure-commandline
```

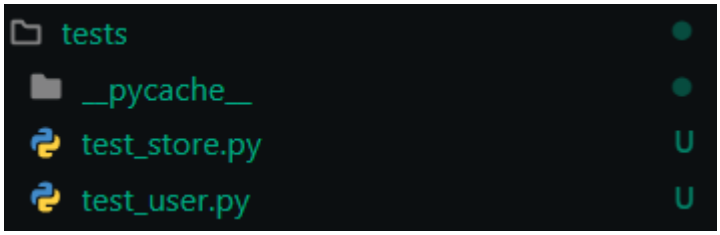
```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> pip install pydantic pytest allure-pytest
```

Был создан файл models.py, в котором были реализованы Pydantic модели для сущностей User и Store, с помощью которых в дальнейшем проводилась валидация данных и авто генерация JSON для PUT и POST запросов.

```python
from pydantic import BaseModel

class UserModel(BaseModel):
    id: int
    username: str
    firstName: str
    lastName: str
    email: str
    password: str
    phone: str
    userStatus: int

class OrderModel(BaseModel):
    id: int
    petId: int
    quantity: int
    shipDate: str
    status: str
    complete: bool
```

Была создана папка tests/, в которой были созданы файлы с тестами (с использованием pytest) для каждой из сущностей - файлы test_user.py и test_store.py соответственно.

Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич



Для каждого запроса к API была добавлена возможность прикрепления JSON ответа к отчету Allure через функцию allure.attach().

```python
import pytest
import allure
from base_request import BaseRequest
from models import OrderModel

BASE_URL = "https://petstore.swagger.io/v2"
base_request = BaseRequest(BASE_URL)

order_id = 9999

@allure.feature("Store")
@allure.story("Create Order")
def test_create_order():
    order_body = OrderModel(
        id=order_id,
        petId=1,
        quantity=1,
        shipDate="2026-02-25T10:00:00.000Z",
        status="placed",
        complete=True
    ).model_dump()
    response = base_request.post("store/order", "", order_body)
    allure.attach(str(response), name="POST Response", attachment_type=allure.attachment_type.

    # Проверка через GET
    order_info = base_request.get("store/order", order_id)
    allure.attach(str(order_info), name="GET Response", attachment_type=allure.attachment_type
    assert order_info["id"] == order_id


@allure.feature("Store")
@allure.story("Delete Order")
def test_delete_order():
    base_request.delete("store/order", order_id)
    response = base_request.get("store/order", order_id, expected_error=True)
    allure.attach(str(response), name="GET Response after Delete", attachment_type=allure.atta
    assert response is not None
```

После чего были запущенны тесты при использовании pytest, которые выполнились успешно.

```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> python -m pytest tests/ --alluredir=allure-results
============================================ test session starts ============================================
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\esketi\Documents\GitHub\ithub-testing
plugins: allure-pytest-2.15.3
collected 5 items

tests\test_store.py ..                                                                              [ 40%]
tests\test_user.py ...                                                                              [100%]

============================================ 5 passed in 25.48s =============================================
```

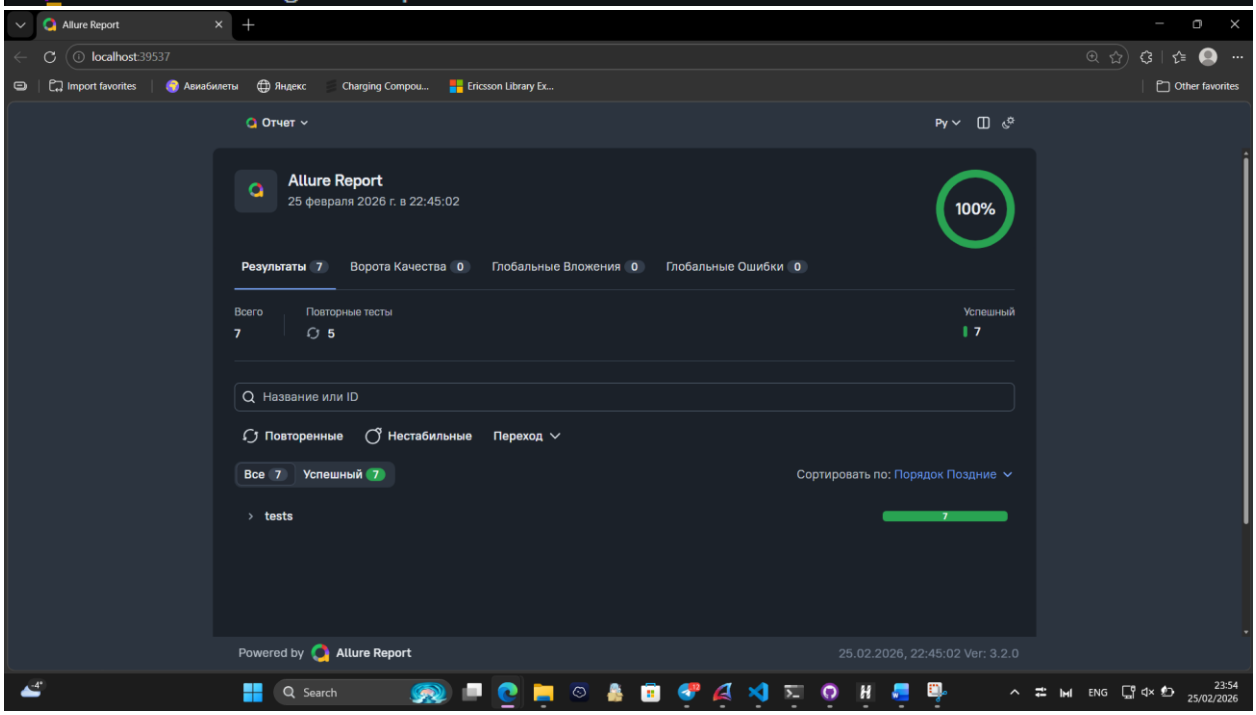
Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

По окончанию выполнения тестов была запущенна генерация отчета с использованием Allure.

```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> npx allure generate allure-results -o allure-report
```

По окончанию генерации отчета от Alure, я открыл его.

```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> npx allure open allure-report
Allure is running on http://localhost:39537
```



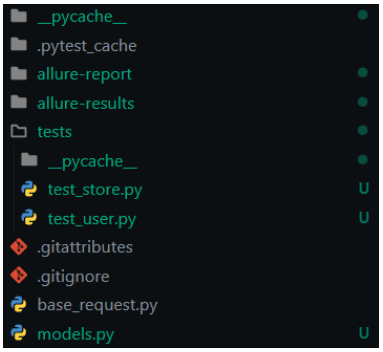
В результате выполнения данной КТ был получен HTML-отчет от Allure, в котором отображались статусы тестов, прикрепленные ответы сервера и подробная информация по выполнению тестов.

Примечание: POST и PUT запросы проверялись через GET, так как API не возвращает объект после выполнения этих операций.

Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

КТ № 6.

В рамках выполнения данной контрольной точки была проведена работа по анализу покрытия кода тестами, созданными в КТ №5.



Для реализации метрик покрытия был установлен дополнительный пакет pytest-cov через pip install.

```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> pip install pytest-cov
```

Далее, тесты запускались через pytest с использованием ранее установленного дополнительного пакета pytest-cov. В результате процент кода, покрытого тестами, составил 99%.

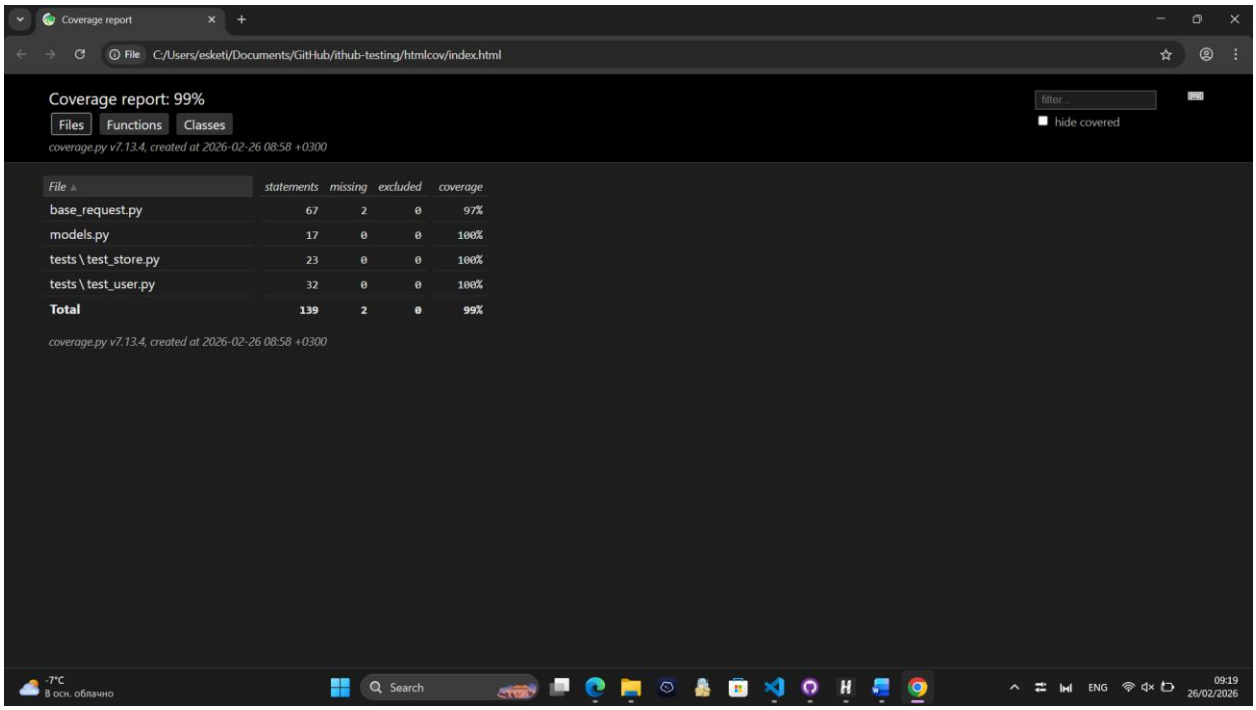
```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> python -m pytest tests/ --cov=.
============================================= test session starts =============================================
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\esketi\Documents\GitHub\ithub-testing
plugins: allure-pytest-2.15.3, cov-7.0.0
collected 5 items

tests\test_store.py ..                                                                                   [ 40%]
tests\test_user.py ...                                                                                   [100%]

================================================ tests coverage ================================================
_________________________________ coverage: platform win32, python 3.13.7-final-0 _________________________________

Name                    Stmts   Miss  Cover
-----------------------------------------
base_request.py            67      2    97%
models.py                  17      0   100%
tests\test_store.py        23      0   100%
tests\test_user.py         32      0   100%
-----------------------------------------
TOTAL                     139      2    99%
============================================= 5 passed in 29.70s ==============================================
PS C:\Users\esketi\Documents\GitHub\ithub-testing>
```

Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

Для более детального анализа покрытия кода тестами был выполнен повторный запуск тестов с генерацией HTML-отчета от cov.

```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> python -m pytest tests/ --cov=. --cov-report=html
================================================ test session starts ================================
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\esketi\Documents\GitHub\ithub-testing
plugins: allure-pytest-2.15.3, cov-7.0.0
collected 5 items

tests\test_store.py ..
tests\test_user.py ...

================================================ tests coverage =====================================
_________________________________ coverage: platform win32, python 3.13.7-final-0 __________________

Coverage HTML written to dir htmlcov
================================================ 5 passed in 27.87s =================================
PS C:\Users\esketi\Documents\GitHub\ithub-testing>
```

В результате был создан подробный HTML-отчет от cov.



На основании полученных данных был проведен анализ полноты тестирования. Результаты показали, что основные методы работы с сущностями User и Store покрыты тестами, включая операции создания, получения, изменения и удаления записей.

Таким образом, выполнение КТ6 позволило подтвердить корректность реализованных тестов и оценить уровень покрытия кода, что является важным этапом в обеспечении качества программного продукта.

Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

КТ № 7

В рамках выполнения КТ №7 была проведена настройка отображения тестов в отчете Allure с использованием специальных декораторов.

Для структурирования отчета в тесты были добавлены следующие декораторы:

1. @allure.feature() — для группировки тестов по сущностям API (User, Store);
2. @allure.story() — для разделения тестов по типам выполняемых операций (GET, POST, PUT, DELETE).

```python
import pytest
import allure
from base_request import BaseRequest
from models import OrderModel

BASE_URL = "https://petstore.swagger.io/v2"
base_request = BaseRequest(BASE_URL)

order_id = 9999

@allure.feature("Store")
@allure.story("Create Order")
def test_create_order():
    order_body = OrderModel(
        id=order_id,
        petId=1,
        quantity=1,
        shipDate="2026-02-25T10:00:00.000Z",
        status="placed",
        complete=True
    ).model_dump()
    response = base_request.post("store/order", "", order_body)
    allure.attach(str(response), name="POST Response", attachment_type=allure.attachment_type.

    # Проверка через GET
    order_info = base_request.get("store/order", order_id)
    allure.attach(str(order_info), name="GET Response", attachment_type=allure.attachment_type
    assert order_info["id"] == order_id


@allure.feature("Store")
@allure.story("Delete Order")
def test_delete_order():
    base_request.delete("store/order", order_id)
    response = base_request.get("store/order", order_id, expected_error=True)
    allure.attach(str(response), name="GET Response after Delete", attachment_type=allure.atta
    assert response is not None
```

Добавление данных декораторов Allure необходимо для структуризации и группировки тестов в интерфейсе Allure.

После их добавления, тесты через pytest были запущенны повторно.

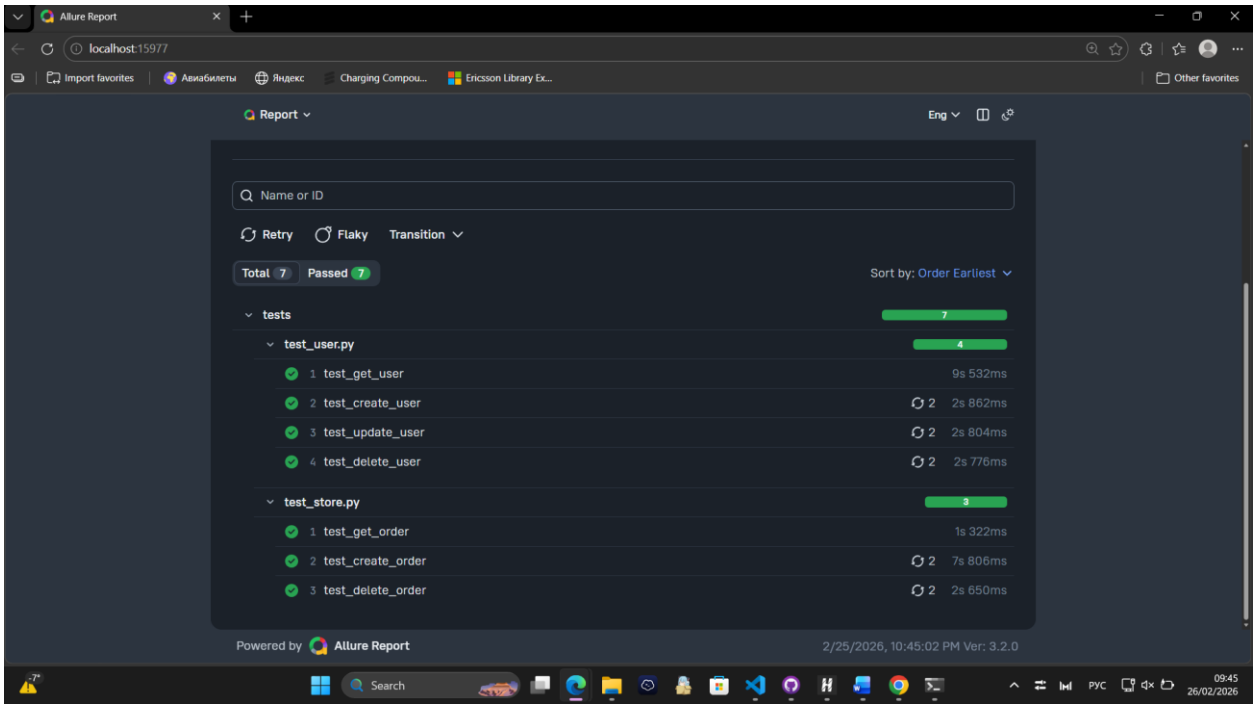
```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> python -m pytest tests/ --alluredir=allure-results
============================================ test session starts ============================================
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\esketi\Documents\GitHub\ithub-testing
plugins: allure-pytest-2.15.3, cov-7.0.0
collected 5 items

tests\test_store.py ..                                                                                  [ 40%]
tests\test_user.py ...                                                                                  [100%]

============================================= 5 passed in 24.02s ============================================
```

Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

Далее, был сгенерирован и в дальнейшем открыт HTML-отчет от Allure.

```
PS C:\Users\esketi\Documents\GitHub\ithub-testing> npx allure generate allure-results -o allure-report
PS C:\Users\esketi\Documents\GitHub\ithub-testing> npx allure open allure-report
Allure is running on http://localhost:15977
```

В конченом итоге был получен HTML-отчет от Allure.



В результате в интерфейсе Allure тесты стали отображаться с разделением по сущностям (User, Store) и по типам операций (GET, POST, PUT, DELETE), что значительно повысило удобство анализа результатов тестирования.

Дисциплина: Тестирование. Проектирование тестов
Контрольная точка: № 4-7
Группа: 3Р1.23
Студент: Байков Даниил Равильевич

ВЫВОД

В ходе выполнения контрольных точек №4-7 по дисциплине Тестирование. Проектирование тестов был произведен полный цикл тестирования API «Petstore».

В частности, изначально был реализован базовый класс для работы с API и написаны тесты для сущностей User и Store (КТ4).

После чего было автоматизировано тестирование с использованием pytest, была обеспечена валидация данных через Pydantic, реализована генерация JSON для POST/PUT запросов, а также прикрепление ответов к отчету Allure (КТ5).

В дальнейшем, было проанализировано покрытие кода тестами с использованием pytest-cov, за счет чего был подтвержден высокий уровень тестового покрытия (99%) и корректность работы всех основных методов API (КТ6).

По окончанию, была настроена визуализация и структурирование тестов в отчете Allure с помощью декораторов @allure.feature и @allure.story, что позволило наглядно группировать тесты по сущностям и типам операций (КТ7).

Таким образом, проделанная работа позволила подтвердить корректность работы API, полноту тестового покрытия и обеспечить удобный визуальный анализ результатов тестирования. В частности, было полученно два полноценных HTML-отчета:

1. HTML-отчет от Allure;
2. HTML-отчет от Pytest-Cov.